

JAVA

05

Software Design Patterns & Project

Write code that scales — master the patterns every professional Java developer lives by.

DURATION	LEVEL	MODULE	UPDATED
10 Sessions	Intermediate	Module 5	June 2026

IN THIS MODULE

01	Understanding Software Design Patterns
02	Implementing MVC + DAO Design Pattern
03	Session Management in Java Web Applications
04	Template Integration (Front-End + Back-End Sync)
05	File & Image Upload / Download
06	Mail Integration (JavaMail API)
07	OTP Verification via Mail Integration
08	Online Payment Integration
09	AJAX & API Integration
10	Introduction to CI/CD & Deployment Workflow

SESSION

01

60 MIN · ARCHITECTURE FUNDAMENTALS

Understanding Software Design Patterns

◆ Learning Objectives

- Explain why design patterns exist and the recurring problems they solve
- Categorise patterns into Creational, Structural, and Behavioural groups
- Implement the Singleton pattern for a shared database connection pool
- Implement the Factory pattern for abstracted object creation
- Describe the DAO pattern and how it separates data access from business logic
- Identify which pattern to apply in a given Java web development scenario

01 Why Design Patterns Exist

Every professional Java codebase faces the same recurring problems: How do you ensure only one database connection object exists? How do you create objects without hardcoding class names? How do you keep controller, business logic, and database code from tangling into a mess? Design patterns are proven, named solutions to these exact problems — solutions refined by experienced developers over decades and documented so teams can communicate in a common vocabulary.

CONCEPT — Gang of Four (GoF)

In 1994, four authors published 'Design Patterns: Elements of Reusable Object-Oriented Software,' cataloguing 23 patterns grouped into 3 families: Creational (how objects are made), Structural (how classes are composed), and Behavioural (how objects communicate). This book — nicknamed the GoF book — is the reference every Java interview draws from. A Pune or Hyderabad IT company job description that says 'knowledge of design patterns' means this book.

Category	Problem It Solves	Key Patterns in This Module
Creational	Control how objects are created	Singleton, Factory
Structural	Compose classes and objects cleanly	DAO (Data Access Object)
Behavioural	Define how objects communicate	MVC (Model-View-Controller)

02 Singleton Pattern — One Instance, Always

Singleton ensures a class has exactly one instance across the entire application and provides a global point of access to it. The most common use case is a database connection manager — you do not want 500 concurrent users each creating their own Connection object, just as Gujarat Cooperative Milk Marketing Federation (Amul) operates one central coordination body, not 500 independent dairy offices. Singleton is that central coordination body for your DB

connection.

```
public class DBConnection {
    private static DBConnection instance;
    private Connection conn;
    private DBConnection() throws Exception {
        conn = DriverManager.getConnection(URL, USER, PASS);
    }
    public static synchronized DBConnection getInstance() throws Exception {
        if (instance == null) instance = new DBConnection();
        return instance;
    }
    public Connection getConnection() { return conn; }
}
```

Singleton class

Single instance holder — static

Actual JDBC connection object

Private: prevents new DBConnection()

Runs only once

Thread-safe accessor

Create only on first call

Always the same object

Expose JDBC connection

PRO TIP — Double-Checked Locking

For high-traffic apps, using synchronized on the whole method is slow. Use double-checked locking: check `instance == null` outside the synchronized block first, then again inside. Add the volatile keyword to the instance field. This is the pattern used in enterprise backends like Flipkart's order service to avoid locking overhead on every single call.

03 Factory Pattern — Create Without Knowing the Class

The Factory pattern delegates object creation to a separate factory class. The caller asks for a Payment object without knowing whether it will receive a UpiPayment, CardPayment, or NetBankingPayment. This is exactly how PhonePe and Paytm handle payment method switching at checkout — the business logic code never changes when a new payment mode is launched, because the factory absorbs all that variability.

```
interface Payment { void processPayment(double amount); }
class UpiPayment implements Payment { ... }
class CardPayment implements Payment { ... }
class NetBankingPayment implements Payment { ... }
class PaymentFactory {
    public static Payment getPayment(String type) {
    }
```

Common contract

Concrete type 1

Concrete type 2

Concrete type 3

The factory

Static factory method

```

        switch (type) {
case "UPI": return new UpiPayment();
    Decision at runtime
case "CARD": return new CardPayment();
default: throw new IllegalArgumentException("Unknown: " + type);
    Fail fast
        }
    }
}

```

04 DAO Pattern — Database Code Belongs in One Place

Data Access Object (DAO) isolates every database CRUD operation into a dedicated layer. Your Servlet calls `UserDAO.save(user)` — it has no idea whether data goes to MySQL, PostgreSQL, or a flat file. This separation makes unit testing trivial (mock the DAO) and database migration painless. Every Spring Boot project you write after this module will have a DAO or Repository layer by default.

<code>interface UserDAO {</code>	Contract — defines what is possible
<code>void save(User user);</code>	Create
<code>User findById(int id);</code>	Read
<code>List<User> findAll();</code>	Read all
<code>void update(User user);</code>	Update
<code>void delete(int id);</code>	Delete
<code>}</code>	
<code>class UserDAOImpl implements UserDAO {</code>	MySQL-specific implementation
<code>// PreparedStatement, ResultSet here — Servlet never sees JDBC</code>	
<code>}</code>	

CASE STUDY — Zerodha Kite Backend Architecture

Context — Zerodha, India's largest discount stockbroker, is based in Bengaluru and serves over 1.3 crore active clients daily with sub-second trade execution.

Challenge — As Zerodha scaled from lakhs to crores of users, mixing database queries with API controller logic made it impossible to switch database drivers or run unit tests without a live DB connection — a critical risk for financial software.

What they did — The team enforced a strict DAO and Repository layer across all backend services, cleanly decoupling JDBC calls from business logic. HikariCP connection pooling replaced ad-hoc connections, implementing the Singleton pattern at the pool level.

Result — Database driver upgrades and schema migrations became zero-downtime operations. Unit test coverage improved drastically because mocked DAOs replaced real DB calls in test suites.

Takeaway — Singleton and DAO patterns are not academic exercises — they are the foundation of every

scalable fintech backend running in India today.

Source: Zerodha Engineering Blog, zerodha.tech/z-connect

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Week 9: Design Patterns and Best Practices, IIT Kharagpur

Link — <https://nptel.ac.in/courses/106105191>

Always link the official channel — beware fan-made or re-upload copies.

WHERE TO GO NEXT

You now know why patterns exist and can implement Singleton, Factory, and DAO independently.

Explore deeper — Read Refactoring.Guru/design-patterns/java for illustrated walkthroughs of all 23 GoF patterns

Try on your own — Rewrite your existing JDBC DBConnection class as a proper Singleton with double-checked locking and volatile

Connect the dots — Next session wires these patterns together into the full MVC + DAO architecture that every servlet-based app uses in production

Going further — Explore Spring's @Bean and @Scope('singleton') to see how the framework enforces Singleton automatically at the container level

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a senior Java architect. Explain Singleton, Factory, and DAO design patterns in simple Hinglish using a Paytm payment app analogy. Tell me when to use each pattern and what mistake beginners make with each. Keep it under 150 words.

SESSION

02

60 MIN · ARCHITECTURE PATTERNS

Implementing MVC + DAO Design Pattern

◆ Learning Objectives

- Draw and explain the complete MVC request-response flow in a Java Servlet-JSP application
- Identify the distinct responsibility of Servlet (Controller), Java Bean (Model), and JSP (View)
- Implement a UserDAO interface and its JDBC-backed implementation class
- Wire MVC and DAO together to build a complete Login and Dashboard feature
- Apply Expression Language (EL) in JSP to display model data without scriptlets
- Explain how this architecture is the direct predecessor of Spring MVC

01 The MVC Pattern — Why Three Layers?

MVC stands for Model-View-Controller. Without it, a beginner's Servlet typically contains JDBC code, if-else business rules, and HTML output all mixed together — a style called the 'Big Ball of Mud.' Think of it like a TOPS placement process: the counsellor (Controller) speaks to the student; the ERP database (Model) holds their profile; the printed offer letter (View) shows the result. Each role has one job, and none of them bleeds into the others.

CONCEPT — MVC Layer Responsibilities

Controller (Servlet): receives the HTTP request, validates input, calls the Model, and chooses which View to render. Model (Java Bean + DAO): carries data between layers and handles all database interaction. View (JSP): renders HTML using data provided by the Controller — zero business logic, zero JDBC. The Controller is the only layer that knows the other two. Model and View never communicate directly.

02 The Controller Layer — Servlet

The Servlet acts like the highway interchange on a Gujarat Expressway — it reads incoming traffic (requests), decides the right lane (service call), and routes each vehicle (response) to the correct destination (JSP). It never processes business logic itself; it only orchestrates between Model and View.

```
@WebServlet("/login")                                Map URL /login to this Servlet
public class LoginServlet extends HttpServlet {
protected void doPost(HttpServletRequest req, HttpServletResponse res)
    Handle POST from login form
throws ServletException, IOException {
String email = req.getParameter("email");
    Read form input
String pass = req.getParameter("password");
```

```

        UserDAO dao = new UserDAOImpl();           No JDBC in Controller
    User user      = dao.findByEmailAndPassword(email, pass);
    Delegate to Model
        if (user != null) {                         Business decision only
    req.getSession().setAttribute("user", user);
    Store in session
    res.sendRedirect("dashboard.jsp");
    Forward to View
        } else {
    req.setAttribute("error", "Invalid credentials");
    Pass error to View
    req.getRequestDispatcher("login.jsp").forward(req, res);
    Back to login
        }
    }
}

```

03 The Model Layer — Bean + DAO

The Model has two parts: the Java Bean (POJO) that carries data between layers, and the DAO class that handles every database operation. The Bean is a pure data container — private fields, public getters and setters, no logic. The DAO is where PreparedStatements live. Keeping them separate means switching from MySQL to PostgreSQL requires changing only UserDAOImpl — nothing else in the application knows or cares.

```

public class User {                               Java Bean — data carrier only
    private int id; private String name; private String email;
    Private fields
        // getters and setters ...                 Encapsulation
}
interface UserDAO {                               Contract
    User findByEmailAndPassword(String email, String password);
    void save(User u);
}
class UserDAOImpl implements UserDAO {            MySQL JDBC implementation
    public User findByEmailAndPassword(String email, String password) {
    // PreparedStatement: SELECT * FROM users WHERE email=? AND password=?
    All JDBC here — nowhere else
    // Map ResultSet to User bean and return
    }
}

```

04 The View Layer — JSP with EL

JSP is the View — it only displays data the Servlet has already placed into request or session scope. A well-written JSP contains zero Java scriptlets: no if-statements inside `<% %>`, no SQL. Use Expression Language (EL) like `${user.name}` to display values cleanly. Think of JSP as the receipt printer at a Reliance Smart store — it prints exactly what the system tells it to, without deciding what to charge.

```

<!-- dashboard.jsp – View only, no Java logic -->
<h2>Welcome, ${user.name}!</h2>
<p>Email: ${user.email}</p>
<c:if test="${not empty error}">
    <p class="error">${error}</p>
</c:if>
<a href='logout'>Logout</a>

```

EL reads session attribute 'user'
No Java code visible in JSP
JSTL conditional (not scriptlet)
Show error set by Servlet
Links to LogoutServlet

05 Complete Login Flow — MVC + DAO End-to-End

Layer	Class / File	Knows About
Controller	LoginServlet.java	Request params, UserDao, JSP path
Model — Data	User.java (Bean)	Its own fields only
Model — DB	UserDaoImpl.java	JDBC, User bean
View	dashboard.jsp	Session attributes via EL only

CASE STUDY — IRCTC Ticket Booking Portal

Context — IRCTC (Indian Railway Catering and Tourism Corporation) runs one of the world's busiest travel portals, serving over 8 lakh ticket bookings per day across India.

Challenge — Early IRCTC versions had business logic and database queries mixed directly inside JSP pages, causing crashes during peak Tatkal booking load and making bug fixes dangerously slow to deploy.

What they did — A multi-tier MVC architecture was adopted: dedicated Servlet controllers for routing, Service + DAO layers for booking logic and DB access, and clean JSP Views. JDBC connection pools replaced ad-hoc Connection objects.

Result — Post-Tatkal booking traffic — spiking to 25,000+ transactions per minute — became manageable. Production release downtime dropped significantly.

Takeaway — MVC + DAO is the minimum professional architecture for any Java web app with real user load.

Source: IRCTC Annual Report 2023-24; NIC India technical infrastructure papers

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Week 10: MVC Architecture with Servlets and JSP, IIT Kharagpur

Link — <https://nptel.ac.in/courses/106105191>

WHERE TO GO NEXT

You can now build a Login and Registration system using proper MVC + DAO architecture on Tomcat.

Explore deeper — Read Oracle's Java EE 8 Tutorial section on Servlets at docs.oracle.com/javaee/7/tutorial/servlets.htm

Try on your own — Add a Registration flow: RegisterServlet -> UserDAOImpl.save(user) -> success.jsp — following the same MVC path

Connect the dots — Next session covers Session Management — how the Controller layer keeps users authenticated across multiple requests

Going further — Explore Spring MVC's DispatcherServlet — it is the production-grade, annotation-driven implementation of exactly this MVC pattern

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java web development coach. Explain the MVC + DAO pattern in Hinglish using an IRCTC ticket booking example. Describe what the Controller, DAO, Bean, and JSP View each do and why they must be separate. Keep it under 150 words.

SESSION

03

60 MIN · WEB APPLICATION SECURITY

Session Management in Java Web Applications

◆ Learning Objectives

- Explain why HTTP is stateless and why session tracking is needed
- Use the HttpSession API to store and retrieve user data across multiple requests
- Implement cookie-based session tracking in a Java Servlet
- Describe hidden form fields and URL rewriting as fallback techniques
- Apply session security best practices including invalidation and timeout
- Compare HttpSession with JWT-based token expiry used in modern REST APIs

01 HTTP Is Stateless — Why That Is a Problem

HTTP is stateless: each request from a browser to the server is completely independent. The server has no memory of previous requests from the same user. Imagine placing an order on Amazon and then clicking 'My Cart' — without session management, the server would not know who you are. Every page load would be like walking into a store where the staff have no idea you were there five seconds ago. This is the core problem session management solves.

02 HttpSession API — Java's Built-In Session Store

HttpSession is the Java EE interface for server-side session management. You create or retrieve the session from `HttpServletRequest`, store objects as attributes using string keys, and retrieve them on any subsequent request. Sessions live in server memory (or a session store like Redis in production) until invalidated or timed out.

```
// In LoginServlet – after successful authentication
HttpSession session = req.getSession(true);    Create session if none exists
session.setAttribute("user", user);            Store User bean in session
session.setAttribute("role", "STUDENT");        Store role string
session.setMaxInactiveInterval(1800);           Expire after 30 minutes of inactivity

// In DashboardServlet – on the next request
HttpSession session = req.getSession(false);    false: do NOT create a new session
if (session == null || session.getAttribute("user") == null) {
    Guard: not logged in
    res.sendRedirect("login.jsp");                Force login
}
User user = (User) session.getAttribute("user");
Retrieve stored object

// In LogoutServlet
```

```
req.getSession().invalidate();
res.sendRedirect("login.jsp");
```

Destroy session — user logged out

TRY THIS LIVE

Add a filter (`javax.servlet.Filter`) that runs before every protected page. Inside, check if `session.getAttribute('user')` is null — if yes, redirect to `login.jsp`. This is the standard authentication guard used in every real Java web app and it is your first taste of middleware.

03 Cookies — Storing the Session ID in the Browser

A cookie is a small key-value pair the server sends to the browser and the browser automatically returns on every subsequent request to that domain. Java's Servlet container uses cookies to persist the `JSESSIONID` automatically — but you can also create your own cookies for things like 'remember me' functionality. Cookies have names, values, expiry, and scope restrictions.

04 Four Session Tracking Techniques Compared

Technique	Where Data Lives	When to Use	Limitation
<code>HttpSession</code>	Server memory / session store	Standard login flow — default choice	Uses server RAM; needs distributed store for clusters
Cookies	Browser (client-side)	Remember-me, preferences	User can delete/disable; size limit ~4KB
URL Rewriting	Appended to every URL <code>?jsessionid=...</code>	When cookies are disabled	Exposes session ID in browser history — avoid
Hidden Form Fields	Inside HTML form as <code><input type=hidden></code>	Single multi-step form flow only	Lost on redirect; not a true session mechanism

05 Session Security Best Practices

Session management is one of the top attack surfaces in web applications. OWASP lists 'Broken Authentication and Session Management' in its Top 10 list every year. Common attacks include session fixation (attacker sets a known session ID before login) and session hijacking (attacker steals the `JSESSIONID` cookie via network sniffing or XSS). Indian banking apps such as SBI YONO implement aggressive session security — timeouts, HTTPS-only cookies, and post-login session regeneration.

WARNING — Never Store Passwords or Card Numbers in Session

`session.setAttribute('password', password)` is a critical security error. Sessions can be serialised to disk, logged, or leaked. Store only the minimal identifier needed — the User ID or a role string. Never store sensitive credentials, card details, or OTPs in session attributes.

```
// Regenerate session ID after login – prevents Session Fixation
HttpSession oldSession = req.getSession(false);
    Get pre-login session (may exist)
if (oldSession != null) oldSession.invalidate();
    Destroy attacker-controlled session
HttpSession newSession = req.getSession(true);
    Fresh session with new ID
newSession.setAttribute("userId", user.getId());
    Store only the ID, not full object

// web.xml – global timeout and HTTPS-only cookie
<session-config>
    <session-timeout>30</session-timeout>          30 minutes idle timeout
<cookie-config><http-only>true</http-only><secure>true</secure></cookie-config>
    Security flags
</session-config>
```

CASE STUDY — SBI YONO Banking Session Security

Context — SBI YONO (You Only Need One) is India's largest digital banking platform, used by over 6 crore registered customers for transactions, loans, and investments.

Challenge — Banking sessions are high-value targets. An idle open session on a shared device at a cyber cafe in Gujarat or Rajasthan exposes full account access to the next user.

What they did — YONO enforces a strict 5-minute inactivity timeout, HTTPS-only JSESSIONID cookies, mandatory session regeneration after login, and a logout-on-back-button behavior using response headers.

Result — Session-based account takeovers on the platform dropped to near zero according to RBI cybersecurity audit reports, making it a benchmark for digital banking security in India.

Takeaway — Correct session management is not optional for any app handling user data — it is a security requirement.

Source: RBI Annual Report on Digital Banking Security 2023-24; SBI YONO documentation

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Week 11: Cookies, Sessions, and Web Security, IIT Kharagpur

Link — <https://nptel.ac.in/courses/106105191>

WHERE TO GO NEXT

You can now implement secure login sessions with proper timeout, invalidation, and session fixation prevention.

Explore deeper — Read OWASP Session Management Cheat Sheet at cheatsheetseries.owasp.org for industry-standard session security rules

Try on your own — Write a `LoginFilter` that checks the session on every protected URL and redirects unauthenticated users to `login.jsp`

Connect the dots — Next session covers Template Integration — plugging a professional Bootstrap admin template into your MVC app so the UI looks production-ready

Going further — Explore Spring Security's session management configuration and its built-in CSRF and Session Fixation protection

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java web security trainer. Explain HTTP session management in Hinglish using an SBI net banking login example. Explain what `JSESSIONID` is, how session fixation works, and how to prevent it. Keep it under 150 words.

SESSION

04

60 MIN · PROFESSIONAL WEB DEVELOPMENT

Template Integration — Front-End and Back-End Sync

◆ Learning Objectives

- Set up a professional web-app folder structure separating static and dynamic resources
- Integrate a free Bootstrap admin HTML template (AdminLTE) into a Java Servlet-JSP project
- Use JSP include directives to share a common header, sidebar, and footer across pages
- Render dynamic database-driven data in a template table using JSTL and EL
- Configure web.xml to serve static assets from the correct paths on Tomcat
- Explain why this folder and layering discipline is enforced in every Spring Boot project

01 Why Folder Structure Matters in Production

A professional Java web app separates two kinds of files sharply: static resources (HTML, CSS, JavaScript, images — things that never change per request) and dynamic resources (JSP, Servlets — things the server processes before sending). Mixing them causes deployment errors, incorrect MIME types, and security leaks. Think of it as a TOPS institute building: the front desk (static HTML/CSS) handles display, and the counsellor's office (Servlet/JSP) handles processing — they occupy different rooms.

<code>src/main/webapp/</code>	Web root — everything under here is deployable
<code>+-- index.jsp</code>	Default landing page
<code>+-- WEB-INF/</code>	Protected — browser cannot access directly
<code>+-- web.xml</code>	Deployment descriptor
<code>+-- views/</code>	JSPs placed here are not publicly accessible
<code>+-- resources/</code>	Publicly accessible static assets
<code>+-- css/adminlte.min.css</code>	Bootstrap / AdminLTE styles
<code>+-- js/adminlte.min.js</code>	Template scripts
<code>+-- img/logo.png</code>	Brand assets
<code>+-- components/</code>	Reusable JSP fragments
<code>+-- header.jsp</code>	Common <head> and top nav
<code>+-- sidebar.jsp</code>	Common navigation sidebar

02 Integrating AdminLTE — Step by Step

AdminLTE is a free, open-source Bootstrap-based admin dashboard template widely used in Indian startup backends. It ships with a sidebar, top navbar, card components, and data tables out of the box. Integration means copying its CSS/JS into your resources folder and replacing its static sample data with live data served from your Servlet and JSP.

TRY THIS LIVE

Download AdminLTE 3 from adminlte.io. Copy `dist/css/adminlte.min.css` and `dist/js/adminlte.min.js` into your `webapp/resources/` folder. Open `starter.html` from AdminLTE, copy its `<head>` and `nav` sections into a `header.jsp` fragment. Include it in your `dashboard.jsp` using `<%@ include file='components/header.jsp' %>`. Refresh in Tomcat and verify the AdminLTE layout appears.

```
<!-- components/header.jsp – reusable header fragment -->
<%@ taglib uri='http://java.sun.com/jsp/jstl/core' prefix='c' %>
    JSTL import
<html><head>
<link rel='stylesheet'
href='${pageContext.request.contextPath}/resources/css/adminlte.min.css'>
    Context-relative path
</head><body class='hold-transition sidebar-mini'>
<nav class='main-header navbar'>                                AdminLTE top bar
<span>Welcome, ${sessionScope.user.name}</span>
    Live user from session
<a href='${pageContext.request.contextPath}/logout'>Logout</a>
</nav>
```

03 Dynamic Content with JSTL and Expression Language

JSTL (JavaServer Pages Standard Tag Library) provides ready-made tags for loops, conditionals, and formatting — replacing messy Java scriptlets inside JSP. Combined with EL (Expression Language), it lets you write JSP that looks almost like HTML. This keeps your View layer clean, readable, and maintainable.

JSTL Tag	Purpose	Example
<code><c:if></code>	Conditional rendering	<code><c:if test='\${user.role == "ADMIN"}></code>
<code><c:forEach></code>	Loop over a collection	<code><c:forEach items='\${students}' var='s'></code>
<code><c:choose></code> / <code><c:when></code>	Switch-like logic	<code><c:when test='\${score > 90}'>Grade A</c:when></code>
<code><c:url></code>	Build context-aware URLs	<code><c:url value='/dashboard/'></code>
<code><fmt:formatNumber></code>	Format currency / numbers	<code><fmt:formatNumber value='\${fee}' type='currency'/></code>

```

<!-- students.jsp – render DB-fetched student list in AdminLTE table -->
<%@ taglib uri='http://java.sun.com/jsp/jstl/core' prefix='c' %>
<table class='table table-bordered'>                                AdminLTE styled table
<thead><tr><th>Name</th><th>Email</th><th>Batch</th></tr></thead>
<tbody>
  <c:forEach items='${students}' var='s'>                          Iterate List<Student> set by Servlet
    <tr>
      <td>${s.name}</td>                                           EL expression
      <td>${s.email}</td>
      <td>${s.batchName}</td>
    </tr>
  </c:forEach>
</tbody>
</table>

```

04 Wiring the Servlet to the Template

The Servlet fetches a List of Student objects from StudentDAOImpl and places it into the request scope using `req.setAttribute('students', studentList)`. Then it forwards to `students.jsp` using `RequestDispatcher`. The JSP never touches the database — the JSTL `<c:forEach>` loop simply iterates the list the Servlet prepared.

```

@WebServlet("/students")
public class StudentListServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
        List<Student> students = new StudentDAOImpl().findAll();
        Fetch from DB
        req.setAttribute("students", students);
        Pass to View
        req.getRequestDispatcher("/WEB-INF/views/students.jsp").forward(req, res);
        Forward to JSP
    }
}

```

CASE STUDY — Gujarat Government e-Governance Portals

Context — The Gujarat Informatics Limited (GIL) manages dozens of citizen-facing web portals including the Digital Gujarat portal used by over 50 lakh citizens for certificates, scholarships, and utility payments.

Challenge — Early portals had inconsistent UI across departments — each team built their own header, footer, and navigation from scratch, causing maintenance nightmares and brand inconsistency.

What they did — GIL adopted a shared JSP template system with common header, footer, and sidebar

fragments included via JSP `<%@ include %>` directives. Static CSS and image assets were centralised in a shared CDN path.

Result — A UI change to the common header (like adding a new ministry logo) propagated to all 30+ portals by updating one file, reducing update time from weeks to minutes.

Takeaway — Template integration with JSP includes and shared static assets is the direct equivalent of a design system — essential for any multi-page application.

Source: Digital Gujarat Portal, digitalgujarat.gov.in; GIL Annual Technology Report 2023

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Week 12: JSP, JSTL, and Dynamic Web Page Generation, IIT Kharagpur

Link — <https://nptel.ac.in/courses/106105191>

WHERE TO GO NEXT

You can now integrate a professional admin template and render live database data through JSTL and EL in a clean View layer.

Explore deeper — Browse AdminLTE 3 component docs at adminlte.io/docs/3.x — cards, datatables, and charts are ready for your JSTL loops

Try on your own — Add an Ahmedabad-specific student count card in AdminLTE using a stat card component, fetching the number from `StudentDAO.countByCity('Ahmedabad')`

Connect the dots — Next session covers File and Image Upload — the next most common feature in any student or employee management portal

Going further — Explore Spring Boot Thymeleaf templates, which replace JSP+JSTL with a modern HTML-first templating engine used in all new Spring Boot projects

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java full stack trainer. Explain in Hinglish how to integrate an AdminLTE Bootstrap template with a Java Servlet-JSP project. What is the folder structure, how do JSP includes work, and how does JSTL `<c:forEach>` render a list of students from the database? Keep it under 150 words.


```

)
public class FileUploadServlet extends HttpServlet {
protected void doPost(HttpServletRequest req, HttpServletResponse res)
throws ServletException, IOException {
String name = req.getParameter("studentName");
    Regular text part
Part photo = req.getPart("profilePhoto");
    File part
String submittedName = photo.getSubmittedFileName();
    Original name from browser
String ext = submittedName.substring(submittedName.lastIndexOf('.'));
    e.g. .jpg
String safe = UUID.randomUUID().toString() + ext;
    Unique, safe file name
String path = req.getServletContext().getRealPath("/uploads/") + safe;
    Server-side save path
    photo.write(path);                                Write file to disk
// Save 'safe' filename to DB for later retrieval
}
}

```

03 File Path vs Database BLOB — Which to Choose?

There are two strategies for storing uploaded files. Storing the file on the server filesystem and saving only the file path in the database is simple and fast for retrieval. Storing the file as a BLOB (Binary Large Object) directly in the database keeps everything in one place but bloats the DB and slows down backups. For most Indian startup apps and educational portals, the file-path strategy with a dedicated storage folder (or AWS S3 in production) is the correct choice.

04 File Validation — Never Trust the Browser

Browser-side HTML accept='image/*' is cosmetic — it does not prevent a malicious user from uploading a PHP file renamed as photo.jpg. Always validate file type on the server by checking the actual MIME type from the Part object, not just the extension. Also enforce a maximum file size on the server side independently of the HTML maxlength attribute.

05 Download Servlet — Streaming Files to the Browser

A Download Servlet reads a file from the server directory and writes it to the HTTP response's output stream with the correct content-disposition header. This header tells the browser whether to display the file inline or prompt the user to save it. Always set Content-Type correctly so the browser knows how to handle the file.

```
@WebServlet("/download")
public class FileDownloadServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
        String filename = req.getParameter("file");
        // UUID filename from DB lookup
        String path = req.getServletContext().getRealPath("/uploads/") + filename;
        File file = new File(path);
        res.setContentType("application/octet-stream");
        // Force download dialog
        res.setHeader("Content-Disposition", "attachment; filename=" + filename);
        res.setContentLength((int) file.length());
        // Tell browser total size
        try (FileInputStream fis = new FileInputStream(file);
            OutputStream os = res.getOutputStream()) {
            byte[] buffer = new byte[4096];
            int bytesRead;
            while ((bytesRead = fis.read(buffer)) != -1) os.write(buffer, 0, bytesRead);
            // Stream chunks
        }
    }
}
```

WARNING — Path Traversal Attack

If the filename parameter is taken directly from the URL without sanitisation, an attacker can pass `'../../etc/passwd'` and download server config files. Always validate that the resolved file path starts with your expected upload directory. Use `filename.replaceAll("[^a-zA-Z0-9._]", "")` to strip dangerous characters before building the path.

CASE STUDY — DigiLocker Document Upload System

Context — DigiLocker, launched by the Ministry of Electronics and Information Technology, stores over 600 crore documents for more than 26 crore registered Indians, including Aadhaar, PAN, mark sheets, and driving licences.

Challenge — Millions of users upload government-issued documents in various formats (PDF, JPEG, PNG). The system must validate format, prevent malicious uploads, generate unique identifiers, and ensure documents are downloadable on demand years later.

What they did — DigiLocker uses server-side MIME validation, UUID-based storage keys, file-path storage with cloud object storage (not DB BLOBs), and signed download URLs with expiry times.

Result — The system handles over 10 lakh document upload and retrieval events daily with near-zero storage corruption or malicious file incidents.

Takeaway — UUID naming, MIME validation, and path-based storage are not just best practices — they are

the architecture of India's national document infrastructure.

Source: DigiLocker Technical Architecture, digilocker.gov.in; MeitY Annual Report 2023-24

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Week 8: File I/O and Streams, IIT Kharagpur

Link — <https://nptel.ac.in/courses/106105191>

WHERE TO GO NEXT

You can now build a complete profile image upload flow with MIME validation, UUID naming, and a working Download Servlet.

Explore deeper — Read the AWS S3 Java SDK docs to see how production apps replace the local file system with S3 for scalable file storage

Try on your own — Add a student profile page that shows the uploaded photo using an `<img>` tag with the `src` pointing to your Download Servlet URL

Connect the dots — Next session covers Mail Integration — learning to send emails (like confirmation with attached documents) from Java using JavaMail

Going further — Explore Spring Boot's `MultipartFile` abstraction and AWS S3 presigned URLs, which replace the manual Download Servlet in production systems

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java backend mentor. Explain in Hinglish how file upload works in Java Servlet using `@MultipartConfig` and the `Part` API. Give an example of uploading a student's Aadhaar card and explain why UUID filenames and MIME validation are important. Keep it under 150 words.

SESSION

06

60 MIN · BACKEND INTEGRATIONS

Mail Integration — JavaMail API

◆ Learning Objectives

- Explain how SMTP works and how Java uses it to send email
- Add the JavaMail Maven dependency and configure an SMTP session with Gmail
- Send a plain-text and an HTML-formatted email from a Servlet
- Attach a file or PDF to an outgoing email using MimeMultipart
- Store SMTP credentials securely using environment variables or a properties file
- Differentiate transactional email (triggered by user action) from marketing email

01 How Email Works — SMTP in 60 Seconds

When your Java application sends an email, it does not talk to the recipient's inbox directly. It speaks SMTP (Simple Mail Transfer Protocol) to a mail server — like Gmail SMTP or a corporate mail relay — which then queues and delivers the message. SMTP runs on port 587 (with STARTTLS encryption) for authenticated sending. Think of it as handing a letter to a courier (the SMTP server) rather than driving it to the recipient's house yourself.

02 JavaMail Setup — Dependency and SMTP Session

Add the Jakarta Mail (formerly JavaMail) dependency to your Maven pom.xml. Then configure a javax.mail.Session object with SMTP host, port, and authentication properties. The Session object represents your authenticated connection channel to the mail server — reuse it across multiple emails instead of creating a new one per send.

```
<!-- pom.xml dependency -->
<dependency>
  <groupId>com.sun.mail</groupId>
  <artifactId>jakarta.mail</artifactId>    JavaMail implementation
  <version>2.0.1</version>
</dependency>
```

```
Properties props = new Properties();           SMTP configuration
props.put("mail.smtp.host", "smtp.gmail.com");
    Gmail SMTP server
props.put("mail.smtp.port", "587");           STARTTLS port
props.put("mail.smtp.auth", "true");          Require authentication
props.put("mail.smtp.starttls.enable", "true");
    Encrypt the connection
```

```
String fromEmail = System.getenv("SMTP_USER");
    From environment variable — NOT hardcoded
String password = System.getenv("SMTP_PASS");
    App password from Google account

Session mailSession = Session.getInstance(props, new Authenticator() {
protected PasswordAuthentication getPasswordAuthentication() {
return new PasswordAuthentication(fromEmail, password);
}
});
```

WARNING — Never Hardcode SMTP Credentials in Source Code

Writing your Gmail password directly in the Java file means it ends up in your Git repository — visible to anyone with repo access. Use `System.getenv('SMTP_PASS')` to read from an environment variable, or store credentials in a properties file excluded from version control via `.gitignore`. AWS SES, Mailgun, and SendGrid provide API keys instead of passwords — even safer.

03 Sending Plain-Text and HTML Emails

`MimeMessage` is the email object you construct and send. For plain text, set the content directly. For HTML emails with branding and formatting — like the registration confirmation emails Swiggy, Zomato, and TOPS send — use `setContent` with MIME type `'text/html; charset=utf-8'`. HTML emails can embed logos, tables, and call-to-action buttons.

```
MimeMessage message = new MimeMessage(mailSession);
    Create the email object
message.setFrom(new InternetAddress(fromEmail, "TOPS Technologies"));
    Sender display name
message.setRecipient(Message.RecipientType.TO, new InternetAddress(toEmail));
    Recipient
message.setSubject("Welcome to TOPS — Registration Confirmed!");
    Subject line

String html = "<h2>Welcome, " + studentName + "!</h2>" +
    HTML body
"<p>Your batch <b>" + batchName + "</b> starts on " + startDate + ".</p>" +
"<p style='color:#E63946;'>Report to TOPS Ahmedabad centre by 9 AM.</p>";

message.setContent(html, "text/html; charset=utf-8");
    Set HTML content type
Transport.send(message);
    Send via configured SMTP
```

04 Sending Emails with Attachments

MimeMultipart lets you combine a text or HTML body with one or more file attachments in a single email. You create separate MimeBodyPart objects — one for the body content and one per attachment — then combine them into a Multipart object. This is how invoices, fee receipts, and hall tickets are emailed to students and professionals across India.

```
MimeMultipart multipart = new MimeMultipart();
    Container for body + attachments

MimeBodyPart textPart = new MimeBodyPart();    Email body part
textPart.setContent(html, "text/html; charset=utf-8");
multipart.addBodyPart(textPart);                Add body first

MimeBodyPart attachPart = new MimeBodyPart();    Attachment part
attachPart.attachFile(new File("/uploads/" + receiptFilename));
    Attach PDF fee receipt
attachPart.setFileName("FeeReceipt_" + studentName + ".pdf");
    Friendly filename for recipient
multipart.addBodyPart(attachPart);                Add attachment

message.setContent(multipart);                    Set combined content
Transport.send(message);
```

05 Transactional vs Marketing Email

CASE STUDY — MakeMyTrip Booking Confirmation Email

Context — MakeMyTrip is India's leading online travel company, headquartered in Gurugram, processing over 1 lakh flight and hotel bookings daily.

Challenge — Every booking must trigger an instant confirmation email with the itinerary, PNR number, and PDF e-ticket attached — even during peak holiday periods when booking volumes spike 10x.

What they did — MakeMyTrip decoupled email sending from the booking transaction using an async message queue (similar to RabbitMQ). An email microservice consumes booking events and calls AWS SES (which uses the same SMTP interface as JavaMail) to dispatch branded HTML emails with PDF attachments.

Result — Confirmation emails are delivered within seconds of booking across 99.9% of transactions, even during Diwali and IPL season traffic spikes.

Takeaway — Start with synchronous JavaMail in your Servlet — but in production, always move email sending to an async queue so a slow mail server never delays the user's response.

Source: *MakeMyTrip Engineering Blog*, medium.com/makemytrip-engineering

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Week 7: Networking and Internet Programming, IIT Kharagpur

Link — <https://nptel.ac.in/courses/106105191>

WHERE TO GO NEXT

You can now send plain-text, HTML, and attachment emails from a Java Servlet triggered by user actions.

Explore deeper — Read AWS Simple Email Service (SES) docs to see how to replace Gmail SMTP with a production-grade service at docs.aws.amazon.com/ses

Try on your own — Create a generic MailService class with a `sendEmail(String to, String subject, String htmlBody)` method reusable across any Servlet in the project

Connect the dots — Next session builds directly on this — you will use JavaMail to send a time-limited OTP code for two-factor login authentication

Going further — Explore Spring Boot's JavaMailSender abstraction and Thymeleaf email templates, which replace string concatenation with clean template files

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java backend developer. Explain in Hinglish how to send an HTML email with a PDF attachment using JavaMail API. Use an example of sending a TOPS course fee receipt to a student in Ahmedabad after online payment. Keep it under 150 words.

SESSION

07

60 MIN · AUTHENTICATION

OTP Verification via Mail Integration

◆ Learning Objectives

- Explain what OTP-based two-factor authentication is and why it is needed
- Generate a cryptographically secure 6-digit OTP using Java's SecureRandom
- Store the OTP with a timestamp in the user's HttpSession and send it via JavaMail
- Build a Servlet that validates the submitted OTP against the stored value
- Enforce a configurable expiry window (e.g. 5 minutes) and single-use invalidation
- Apply OTP security best practices including rate limiting awareness and non-disclosure

01 Why OTP? — Two-Factor Authentication

A password alone is a single factor — if it is leaked, guessed, or phished, an attacker has full access. Two-Factor Authentication (2FA) adds a second layer: something the user knows (password) plus something the user receives (OTP on email or phone). Every major Indian app — SBI YONO, Zerodha, IRCTC, and Google — requires OTP for high-risk actions like login or large transactions. This session implements email OTP, the simplest form of 2FA you can add to a Java web app.

02 Generating a Secure OTP in Java

Never use `java.util.Random` for OTP generation — it is pseudo-random and predictable. Use `java.security.SecureRandom`, which draws entropy from the OS and is cryptographically secure. A 6-digit OTP has 1 million possible values — with a 5-minute expiry, the effective brute-force window is tiny. For TOPS-level apps, 6 digits is the industry standard; banking apps sometimes use 8.

```
import java.security.SecureRandom;                                Cryptographically secure random

public class OtpUtil {
    private static final SecureRandom random = new SecureRandom();
                                Shared, thread-safe instance

    public static String generate6DigitOtp() {
        int otp = 100000 + random.nextInt(900000);
                                Range: 100000–999999

        return String.valueOf(otp);                                Return as String to preserve leading zeros
    }
}
```

03 Storing OTP with Timestamp — Session vs Database

You need to store the OTP and the time it was generated so you can verify it and check expiry. For a single-server app, storing in HttpSession is simple and fast. For a multi-server or production setup, store in the database (a table with email, otp, created_at, used columns) so all servers share state. For this module, we use the Session approach.

```
// OtpRecord – a simple carrier class
public class OtpRecord {
    public final String otp;                The generated OTP string
    public final long   generatedAt;        System.currentTimeMillis() at generation
    public OtpRecord(String otp) {
        this.otp = otp;
    }
    this.generatedAt = System.currentTimeMillis();
}

// In SendOtpServlet.doPost()
String email    = req.getParameter("email");    User's email from login form
String otp      = OtpUtil.generate6DigitOtp();
    Generate
OtpRecord record = new OtpRecord(otp);          Wrap with timestamp
req.getSession().setAttribute("otpRecord", record);
    Store in session
req.getSession().setAttribute("otpEmail", email);
    Remember which email
MailService.sendOtpEmail(email, otp);           Send via JavaMail
res.sendRedirect("verifyOtp.jsp");              Redirect to OTP entry form
```

04 Sending the OTP via Email

The OTP email should be plain, clear, and urgent. Include the code in large text, state the expiry time clearly, and explicitly say not to share the code. This is the exact pattern used by banks like HDFC and ICICI — a subject line like 'Your OTP: 847291 — Do Not Share' with the expiry in the body.

05 Validating the OTP — Expiry and Single-Use

The Verify OTP Servlet retrieves the stored OtpRecord from the session, checks that the OTP matches, and verifies that less than 5 minutes (300,000 milliseconds) have elapsed since generation. After a successful match, the OTP must be immediately removed from the session so it cannot be reused — even if the same code is submitted again one second later, it must be rejected.

```
// In VerifyOtpServlet.doPost()
String submitted = req.getParameter("otp");    What the user typed
HttpSession sess = req.getSession(false);
OtpRecord record = (OtpRecord) sess.getAttribute("otpRecord");
    Retrieve stored OTP

if (record == null) { /* session expired – redirect to start */ return; }

long elapsed = System.currentTimeMillis() - record.generatedAt;
    Milliseconds since generation
boolean expired = elapsed > 5 * 60 * 1000L;    5-minute window
boolean match = record.otp.equals(submitted.trim());
    String comparison — not ==

sess.removeAttribute("otpRecord");            Invalidate immediately — single use

if (!expired && match) {
    sess.setAttribute("authenticated", true);    Mark session as verified
    res.sendRedirect("dashboard");
} else {
    req.setAttribute("error", expired ? "OTP expired" : "Invalid OTP");
    req.getRequestDispatcher("verifyOtp.jsp").forward(req, res);
}
```

WARNING — OTP Security Non-Negotiables

Never log OTPs in server logs — logs are often stored in plaintext and accessible to admins. Never include the OTP in the URL (no `redirect?otp=847291`). Never allow unlimited retries — after 3 wrong attempts, force regeneration. Do not send OTPs in SMS and email simultaneously to the same user without rate limiting.

CASE STUDY — Aadhaar OTP-Based e-KYC

Context — UIDAI's Aadhaar e-KYC system is used by banks, telecom companies, and government portals across India to verify citizen identity using an OTP sent to the Aadhaar-linked mobile number.

Challenge — The system must generate OTPs for over 10 crore monthly verification requests, ensure each OTP is single-use, expires in 10 minutes, and cannot be replayed even if intercepted.

What they did — UIDAI uses server-side OTP storage with a database table that marks each OTP as 'used' on first successful verification. Expiry is enforced at the database level using a `generated_at` timestamp with a 10-minute TTL. Rate limiting prevents more than 3 OTP requests per Aadhaar number per hour.

Result — Aadhaar OTP-based KYC processes over 3.5 crore authentications per month with a fraud rate below 0.001%.

Takeaway — Single-use, time-limited, rate-limited OTP is the minimum standard — as implemented by India's national identity infrastructure.

Source: UIDAI Annual Report 2023-24; uidai.gov.in technical documentation

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Week 7: Security and Cryptography in Java, IIT Kharagpur

Link — <https://nptel.ac.in/courses/106105191>

WHERE TO GO NEXT

You can now implement a complete email OTP login flow with secure generation, session storage, and expiry validation.

Explore deeper — Read OWASP's Authentication Cheat Sheet at cheatsheetseries.owasp.org for rate limiting, lockout, and OTP lifecycle best practices

Try on your own — Add a 'Resend OTP' button that calls `SendOtpServlet` again — but block resend if the last OTP was generated less than 60 seconds ago

Connect the dots — Next session integrates a real payment gateway — Razorpay — adding the most commercially valuable feature to your Campus Connect Portal

Going further — Explore TOTP (Time-based OTP) using Google Authenticator and the Java TOTP library — the same algorithm used by all authenticator apps

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a Java security instructor. Explain OTP-based login in Hinglish using an Aadhaar e-KYC example. Explain why `SecureRandom` is used instead of `Random`, how expiry is checked using `System.currentTimeMillis()`, and why `removeAttribute()` is important. Keep it under 150 words.

SESSION

08

60 MIN · PAYMENT & API INTEGRATION

Online Payment Integration

◆ Learning Objectives

- Explain the end-to-end payment gateway flow: order creation, checkout, and verification
- Create a Razorpay order on the server side using the Razorpay Java SDK
- Embed the Razorpay JavaScript checkout widget in a JSP page
- Verify payment signature server-side using HMAC-SHA256 to prevent fraud
- Store SMTP and API keys securely using environment variables
- Compare Razorpay, PayU, and CCAvenue for Indian e-commerce use cases

01 Payment Gateways in India — The Landscape

A payment gateway is the technology layer that authorises and processes online payments between a customer's bank and your application. In India, the most widely used gateways are Razorpay (Bengaluru-based, founded 2014, powering over 8 million businesses), PayU, and CCAvenue. Each exposes a REST API your backend calls to create orders, and a JavaScript SDK your frontend uses to show the payment popup. Your code never handles the actual card or UPI credentials — the gateway does.

02 Server-Side Order Creation — Razorpay Java SDK

Before the checkout popup appears, your server must create an order in Razorpay's system. This gives you a Razorpay order ID that links the payment to your internal transaction record. Amount must be passed in paisa (paise) — for Rs 500 pass 50000. Never send amount from the frontend; always calculate it on the server to prevent tampering.

```
<!-- pom.xml -->
<dependency>
  <groupId>com.razorpay</groupId>
  <artifactId>razorpay-java</artifactId>
  <version>1.4.5</version>
</dependency>
```

```
// In CreateOrderServlet.doPost()
String keyId      = System.getenv("RAZORPAY_KEY_ID");
    From Razorpay Dashboard
String keySecret = System.getenv("RAZORPAY_KEY_SECRET");
    Never hardcode in source
```

```

RazorpayClient client = new RazorpayClient(keyId, keySecret);
    Initialise SDK

JSONObject options = new JSONObject();
options.put("amount", 50000);           Rs 500 in paise — server-calculated
options.put("currency", "INR");
options.put("receipt", "receipt_TOPS_" + System.currentTimeMillis());
    Unique receipt ID

Order order = client.orders.create(options);    Call Razorpay API
String razorpayOrderId = order.get("id");       e.g. order_OABcDEfGhIJ123

// Save razorpayOrderId to your DB for later verification
req.setAttribute("razorpayOrderId", razorpayOrderId);
    Pass to JSP
req.setAttribute("razorpayKeyId", keyId);       Frontend needs only key ID, not secret
req.getRequestDispatcher("checkout.jsp").forward(req, res);

```

03 Front-End — Razorpay JavaScript Checkout

The Razorpay JS SDK renders a fully hosted payment popup that handles UPI, cards, netbanking, and wallets. You pass the order ID from the server, configure the amount and name, and set a handler function that receives the payment response on success. The popup is mobile-optimised and compliant with RBI's payment security guidelines.

04 Server-Side Signature Verification — Preventing Fraud

After payment, Razorpay returns three values to the browser: `razorpay_payment_id`, `razorpay_order_id`, and `razorpay_signature`. The signature is an HMAC-SHA256 hash of `order_id + '|' + payment_id`, signed with your key secret. Your server recomputes this hash independently and compares it to the received signature. If they match, the payment is genuine. Never rely solely on the `payment_id` from the frontend — a fraudster can manipulate it.

```

// In VerifyPaymentServlet.doPost()
String orderId    = req.getParameter("rzpOrderId");
String paymentId  = req.getParameter("rzpPaymentId");
String signature  = req.getParameter("rzpSignature");

String payload = orderId + "|" + paymentId;    Data to hash
String secret  = System.getenv("RAZORPAY_KEY_SECRET");

Mac hmac = Mac.getInstance("HmacSHA256");
hmac.init(new SecretKeySpec(secret.getBytes(), "HmacSHA256"));

```

```
String computed = bytesToHex(hmac.doFinal(payload.getBytes()));
```

Server-computed hash

```
if (computed.equals(signature)) {           Genuine payment — signatures match
// Mark order PAID in DB, send confirmation email
    res.sendRedirect("paymentSuccess.jsp");
} else {
    res.sendRedirect("paymentFailure.jsp");   Signature mismatch — reject
}
```

WARNING — Never Trust Frontend Payment Confirmation Alone

A frontend hack can send a fake payment_id claiming payment was successful. Always verify the Razorpay HMAC signature on the server before crediting the user's account or issuing a receipt. This is RBI's mandated requirement for payment processors, and Razorpay's own integration documentation marks signature verification as mandatory.

05 Choosing the Right Gateway — Quick Comparison

Gateway	Setup Ease	UPI Support	Best For	Pricing (MDR)
Razorpay	Very easy — sandbox in minutes	Yes (all modes)	Startups, edu-tech, SaaS	2% + GST (standard)
PayU	Moderate	Yes	Mid-size e-commerce	1.99% standard
CCAvenue	Complex — paperwork heavy	Yes	Enterprise retail, travel	Custom negotiated
Instamojo	Easy	Yes	Solopreneurs, small stores	2% + Rs 3 flat fee

CASE STUDY — Amul Online Store Payment Integration

Context — Amul, India's largest dairy cooperative based in Anand, Gujarat, launched amul.com for direct-to-consumer gifting and bulk orders, serving customers across India.

Challenge — Amul's online store needed to support UPI, cards, and netbanking while ensuring zero fraud and instant order confirmation for perishable dairy gift orders.

What they did — Razorpay was integrated for order creation and checkout. Server-side HMAC signature verification gates every order status update. On successful verification, an email confirmation with an invoice PDF is sent via JavaMail before the order enters the fulfilment queue.

Result — Online store transactions increased from 0 to over 5,000 per day within three months of launch,

with zero payment fraud incidents reported in the first year.

Takeaway — HMAC signature verification + instant email confirmation is the minimum viable payment integration for any Indian e-commerce app.

Source: Amul.com launch coverage, Economic Times 2021; Razorpay case study blog

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Week 7: Networking, REST APIs, and HTTP in Java, IIT Kharagpur

Link — <https://nptel.ac.in/courses/106105191>

WHERE TO GO NEXT

You can now build an end-to-end payment flow: server-side order creation, Razorpay JS checkout, and HMAC signature verification.

Explore deeper — Read Razorpay's official Java integration guide at razorpay.com/docs/payments/server-integration/java for webhook setup

Try on your own — After successful payment verification, send a fee receipt PDF as an email attachment using the JavaMail code from Session 06

Connect the dots — Next session covers AJAX — enabling real-time interactions like live email checks without full page reloads, the foundation of modern UX

Going further — Explore Razorpay Webhooks — a server-to-server callback Razorpay fires for every payment event, more reliable than browser callbacks for critical payments

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a payment integration expert. Explain in Hinglish how Razorpay payment integration works in a Java web app. Describe the three steps: order creation on server, JS checkout popup, and HMAC signature verification. Use a TOPS course fee payment as the example. Keep it under 150 words.

SESSION

09

60 MIN · MODERN WEB INTERACTIONS

AJAX and API Integration

◆ Learning Objectives

- Explain what AJAX is and how it enables asynchronous browser-server communication
- Use the Fetch API to send GET and POST requests from a JSP page without page reload
- Return JSON from a Java Servlet and parse it in JavaScript
- Build a real-time email availability check using AJAX and a Servlet
- Handle errors gracefully in both the Servlet response and the JavaScript handler
- Understand how this bridges to Axios in React — the pattern used in the next module

01 What Is AJAX — And Why It Changed the Web

AJAX (Asynchronous JavaScript and XML) is a technique that lets the browser send a request to the server and receive a response without reloading the entire page. Before AJAX, every button click or form submit caused a full-page refresh — jarring and slow. AJAX is why Swiggy can show you real-time delivery tracking, why Google Suggest shows options as you type, and why your TOPS registration form can check if your email is already taken before you click Submit.

02 Returning JSON from a Java Servlet

When a Servlet serves an AJAX request, it does not return a full HTML page — it returns data, typically in JSON format. You set the response Content-Type to application/json, write a JSON string to the output, and call return immediately. No RequestDispatcher, no forward to JSP. The JavaScript on the browser side parses this JSON and updates the UI without reloading.

```
// CheckEmailServlet.doGet() — responds with JSON
res.setContentType("application/json");           Tell browser this is JSON
res.setCharacterEncoding("UTF-8");

String email = req.getParameter("email");         Email from AJAX request
boolean taken = new UserDAOImpl().existsByEmail(email);
// DB lookup

// Build JSON manually (or use Jackson/Gson for complex objects)
String json = "{\"taken\": " + taken + "}";        e.g. {"taken": true}
res.getWriter().write(json);                     Write to response — no forward
// No RequestDispatcher — AJAX responses end here
```

03 Fetch API — Calling the Servlet from JavaScript

The Fetch API call on the JSP page targets the Servlet URL, includes query parameters or a request body, and processes the JSON response in a `then()` callback (or `await` in an `async` function). Keep your Fetch logic in a clean, named function — not inlined in event handlers — so it is easy to read and reuse.

```
<!-- registration.jsp - email input with real-time check -->
<input type='email' id='emailField' name='email'
oninput='checkEmail(this.value)' placeholder='your@email.com' />
    Trigger on every keystroke
<span id='emailMsg'></span>                                Feedback display element

<script>
async function checkEmail(email) {
    if (email.length < 5) return;                            Skip very short inputs
    const res  = await fetch('/checkEmail?email=' + encodeURIComponent(email));
    Call Servlet
    const data = await res.json();                             Parse JSON response
    const msg  = document.getElementById('emailMsg');
    if (data.taken) {                                          Use response data
        msg.textContent = '[x] Email already registered';
        msg.style.color = 'red';
    } else {
        msg.textContent = '[ok] Email available';
        msg.style.color = 'green';
    }
}
</script>
```

TRY THIS LIVE

Add the `CheckEmailServlet` and the `oninput` handler to your registration form. Pre-populate the users table with a few emails. Open the registration form in the browser and type one of those emails — you should see 'Email already registered' appear instantly without any page reload. Then type a new email and see 'Email available'.

04 Sending Data with POST via Fetch

For operations that modify data (submitting a form asynchronously, adding a comment, updating a status), use an HTTP POST Fetch request with a JSON body. The Servlet reads the raw JSON from the request input stream and parses it using Jackson ObjectMapper or Gson. Set `Content-Type: application/json` on the request — the Servlet needs this to know the body format.

05 Error Handling and the Bridge to Axios in React

Always handle errors in AJAX calls — network failures, 4xx status codes, and server exceptions should show user-friendly messages, not silent failures. The `fetch()` Promise only rejects on network errors; HTTP 4xx/5xx responses still resolve as a response object. Always check `response.ok` before processing the body. This same pattern — check status, parse JSON, update UI — is exactly what React's Axios library does, just with cleaner syntax.

```
async function checkEmail(email) {
  try {
    // Wrap in try-catch for network errors
    const res = await fetch('/checkEmail?email=' + encodeURIComponent(email));
    if (!res.ok) throw new Error('Server error: ' + res.status);
    // Check HTTP status
    const data = await res.json();
    // ... update UI
  } catch (err) {
    console.error('AJAX error:', err);
  }
  document.getElementById('emailMsg').textContent = 'Connection error. Try again.';
}

// React/Axios equivalent (for awareness – same concept)
const { data } = await axios.get('/api/checkEmail', { params: { email } });
// Axios auto-parses JSON
```

CASE STUDY — Swiggy Real-Time Search and Menu Filter

Context — Swiggy, India's largest food delivery platform based in Bengaluru, handles over 3 million daily food orders across 500+ cities.

Challenge — Showing restaurant search results and filtering menu items with full-page reloads would make the browsing experience unbearably slow on Indian 4G connections, driving users to abandon the app.

What they did — Swiggy uses AJAX-based search and filter throughout the app. As a user types in the search bar, a debounced AJAX call fetches matching restaurants from the backend API, which responds with a JSON array that JavaScript renders into cards — no page reload.

Result — Average search-to-order time dropped by 40% compared to page-reload-based equivalents, contributing to higher conversion rates on the platform.

Takeaway — AJAX with debouncing and JSON API responses is the foundation of every high-conversion Indian consumer app.

Source: Swiggy Engineering Blog, bytes.swiggy.com; company product analytics 2023

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Programming in Java — Week 12: REST Web Services and JSON in Java, IIT Kharagpur

Link — <https://nptel.ac.in/courses/106105191>

WHERE TO GO NEXT

You can now build real-time interactive features — live validation, async form submission, and dynamic content updates — using Fetch and JSON Servlets.

Explore deeper — Read MDN's Fetch API guide at developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch for all options including headers, body, and AbortController

Try on your own — Add a live character counter for the 'About Me' textarea that uses AJAX to check if a username is already taken in the Campus Connect Portal

Connect the dots — Next session is the final in this module — CI/CD and Deployment — learning to push your complete Campus Connect Portal to the public internet automatically

Going further — Explore the Jackson ObjectMapper's full API for serialising and deserialising nested Java objects to and from JSON, which is the foundation of Spring Boot REST

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a JavaScript and Java backend trainer. Explain in Hinglish how AJAX works using the Fetch API. Use a Swiggy-style real-time restaurant search as an example. Explain the flow: user types, Fetch calls Servlet, Servlet returns JSON, JavaScript updates the DOM. Keep it under 150 words.

SESSION

10

60 MIN · DEVOPS FUNDAMENTALS

Introduction to CI/CD and Deployment Workflow

◆ Learning Objectives

- Explain what CI/CD means and the problem it solves in team-based Java projects
- Understand the stages of a CI pipeline: build, test, and deploy
- Write a basic GitHub Actions workflow YAML to automate build and test on every push
- Write a simple JUnit 5 unit test for a DAO or service class
- Deploy a Spring Boot JAR to Render or a basic EC2 instance
- Understand the professional standard of 'failing fast' — blocking a bad deployment before it reaches production

01 What Is CI/CD — And Why Every Team Uses It

CI/CD stands for Continuous Integration and Continuous Delivery (or Deployment). Without it, deploying a Java web app to production means a developer manually building a WAR or JAR, FTPing it to a server, restarting Tomcat, and hoping nothing breaks. With CI/CD, every Git push automatically triggers a pipeline that builds the app, runs tests, and deploys if everything passes. Think of it as the QC belt at an Ahmedabad textile factory: every metre of fabric passes the same quality checks automatically before leaving the floor.

02 GitHub Actions — Your CI Pipeline in a YAML File

GitHub Actions is a free CI/CD platform built into GitHub. You define your pipeline as a YAML file inside `.github/workflows/` in your repository. A workflow is a set of jobs that run on GitHub's cloud servers (called runners) whenever a specified trigger occurs — such as a push to the main branch. Each job runs steps: checkout code, set up Java, run Maven build, run tests.

```
# .github/workflows/ci.yml
name: Java CI with Maven                                Workflow name shown in GitHub UI

on:                                                       Trigger definition
  push:
    branches: [ main ]                                   Run on every push to main
  pull_request:
    branches: [ main ]                                   Run on PRs targeting main

jobs:
  build-and-test:                                         Job name
    runs-on: ubuntu-latest                               Free Linux runner
    steps:
      - uses: actions/checkout@v4                       Clone the repository
```

```

- uses: actions/setup-java@v4           Install Java
with: { java-version: '17', distribution: 'temurin' }
Java 17 LTS
- run: mvn -B package --file pom.xml    Build and run tests
- run: mvn -B verify                    Run integration tests

```

TRY THIS LIVE

Push your Campus Connect Portal to a GitHub repository. Create the `.github/workflows/ci.yml` file above. Push the file — go to the Actions tab in GitHub and watch your build run live. Intentionally break a test or introduce a compilation error, push again, and see the pipeline go red. This is 'failing fast' in action.

03 JUnit Testing — The Test Layer in Your Pipeline

The CI pipeline is only as valuable as the tests running inside it. JUnit 5 is the standard Java testing framework. A unit test verifies one piece of logic in isolation — without a live database, without a running server. You mock the DAO (using Mockito) to test your Servlet's business logic independently. The CI pipeline runs all tests on every push, so a broken unit test blocks the deployment before any bad code reaches production.

```

<!-- pom.xml — JUnit 5 dependency -->
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.10.0</version>
  <scope>test</scope>           Test scope — not in production build
</dependency>

```

```

// src/test/java/OtpUtilTest.java
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class OtpUtilTest {

    @Test                       JUnit 5 test annotation
    void otpShouldBeSixDigits() {
        String otp = OtpUtil.generate6DigitOtp();
        Call the method under test
        assertEquals(6, otp.length(), "OTP must be exactly 6 digits");
        Assert
    }
}

```

```

assertTrue(otp.matches("\\d{6}"), "OTP must be numeric");
    Assert digits only
}

@Test
void twoOtpsShouldBeUnique() {
    // Statistical uniqueness test
    assertEquals(OtpUtil.generate6DigitOtp(), OtpUtil.generate6DigitOtp());
}
}

```

04 Deployment Targets — Where Your App Goes Live

For a Spring Boot app that packages as an executable JAR, deployment is as simple as running `java -jar app.jar` on a server. The choice of deployment target depends on cost, control, and scale. Render and Railway are zero-config platforms that detect your `pom.xml` and deploy automatically — ideal for portfolios and demos. AWS EC2 gives full server control — the choice for production TOPS or campus projects that need custom configuration.

Platform	Setup Effort	Cost (Free Tier)	Best For	Used By
Render	Very Low — connect GitHub, done	Yes (750 hrs/month)	Student projects, demos, startups	Indie developers, hackathons
Railway	Very Low — zero config	Yes (small credit)	Quick Spring Boot deploys	Startups, TOPS project demos
Heroku	Low — Procfile needed	No (paid only now)	Legacy: many tutorials reference it	Older enterprise tutorials
AWS EC2 + RDS	High — manual setup	12-month free tier	Production apps needing full control	Zerodha, Nykaa, TOPS ERP

```

# Deploy Spring Boot JAR to Render via GitHub Actions
- name: Deploy to Render
  run: |
    curl -X POST '${{ secrets.RENDER_DEPLOY_HOOK }}' \
      -d '{
        "hook_url": "${{ secrets.RENDER_DEPLOY_HOOK }}"
      }'
    # Render deploy hook URL stored as GitHub secret
    Trigger Render to pull latest build

# Dockerfile (optional — for full container deploy)
FROM eclipse-temurin:17-jre-alpine
COPY target/campusconnect-1.0.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app.jar"]

```

Lightweight Java 17 base image

Copy built JAR

Spring Boot default port

Start the app

CASE STUDY — Meesho CI/CD Pipeline at Scale

Context — Meesho, a Bengaluru-based social commerce platform used by over 15 crore customers and 1.3 crore resellers, deploys code to production multiple times per day.

Challenge — With 500+ engineers across teams, manually coordinating deployments and catching integration bugs late in the cycle was causing production incidents that took hours to roll back.

What they did — Meesho adopted GitHub Actions for CI on every pull request, with JUnit and integration test gates. Passing builds are automatically promoted to staging and deployed to production via blue-green deployment on AWS EKS.

Result — Deployment frequency increased from once a week to 15+ times per day. Mean Time to Recovery (MTTR) for production bugs dropped from 4 hours to under 20 minutes because broken changes are caught in the pipeline before reaching users.

Takeaway — CI/CD is not just a DevOps tool — it is the infrastructure that allows teams to move fast without breaking things.

Source: Meesho Engineering Blog, medium.com/meesho-tech; NASSCOM DevOps India Report 2024

WATCH — VIDEO REFERENCE (HINDI/ENGLISH)

Channel — NPTEL (official — IITs / IISc)

Watch — Software Engineering — Lecture on Software Testing, CI/CD, and DevOps Practices, IIT Kharagpur

Link — <https://nptel.ac.in/courses/106105087>

WHERE TO GO NEXT

You can now write a JUnit test, set up a GitHub Actions CI pipeline, and deploy a Spring Boot app to Render automatically on every push.

Explore deeper — Read GitHub Actions documentation at docs.github.com/en/actions for advanced triggers, matrix builds, and secrets management

Try on your own — Add a second GitHub Actions job that only runs after the build-and-test job succeeds — use the needs: build-and-test keyword to chain jobs

Connect the dots — Module 5 is complete. You have built a professional-grade Java web application with design patterns, session security, file handling, payments, AJAX, and automated deployment — the complete Campus Connect Portal

Going further — Explore Docker and Kubernetes basics — containerising your Spring Boot app so it runs identically on any cloud, which is the industry standard for all new Java deployments in India

AI PROMPT — ASK CHATGPT / GEMINI / CLAUDE

Act as a DevOps coach for a Java developer. Explain in Hinglish what CI/CD is and how GitHub Actions

automates the build-test-deploy cycle for a Spring Boot app. Use an example of a TOPS Campus Connect Portal being automatically deployed to Render after every push to the main branch. Keep it under 150 words.

REVIEW



INTERVIEW PREP · MODULE 5

Interview Questions

◆ Software Design Patterns And Project

Rehearse each answer out loud and aim for a 60-second response. Use the hint only as a nudge if you are stuck. Compare your attempt with the model answer afterward to find gaps.

1

What is a software design pattern, and why are design patterns important in Java web development?

Think about code maintainability and avoiding repeated solutions to common problems.

MODEL ANSWER

A design pattern is a reusable, named solution to a recurring problem in software design. Patterns are important because they give developers a shared vocabulary and proven blueprints, reducing the time spent reinventing solutions. In Java web development, patterns like MVC and DAO ensure that concerns are separated cleanly, making codebases easier to test, extend, and hand off to teammates. They also make code reviews faster because the structure is predictable.

2

Explain the difference between creational, structural, and behavioral design patterns.

Consider what aspect of object-oriented design each category addresses.

MODEL ANSWER

Creational patterns deal with object creation — controlling how and when objects are instantiated (e.g., Singleton, Factory). Structural patterns deal with class and object composition — how classes are assembled into larger structures (e.g., Facade, Decorator). Behavioral patterns deal with communication between objects — who is responsible for what and how objects interact (e.g., Observer, Strategy). Knowing which category a pattern belongs to helps you quickly identify it in a codebase and apply the right one for a given problem.

3

What is the Singleton pattern, and why would you use it for a database connection pool in a Java application?

Focus on controlling the number of instances and resource management.

MODEL ANSWER

The Singleton pattern ensures that only one instance of a class is created and provides a global access point to it. For a database connection pool, creating multiple pool instances wastes resources and causes inconsistent state. By making the pool class a Singleton, every part of the application shares the same managed pool. The classic Java implementation uses a private constructor and a static getInstance() method that lazily creates the instance. For thread safety, use double-checked locking or an enum-based Singleton.

4

How does the Factory pattern help in object creation? Give a real-world example relevant to Java web apps.

Think about situations where you want to hide the instantiation logic.

MODEL ANSWER

The Factory pattern delegates object creation to a factory method, hiding the concrete class from the caller. The caller asks the factory for an object by type and gets back an instance without knowing which subclass was constructed. For example, a NotificationFactory could return an EmailNotification, SMSNotification, or PushNotification object based on the user's preference. This makes adding a new notification channel as simple as adding a new class and updating the factory — no changes to the calling code.

5

What is the DAO (Data Access Object) pattern, and how does it help separate concerns in a Java web application?

Consider how data access logic is isolated from business logic.

MODEL ANSWER

The DAO pattern isolates all database interaction code into a dedicated class, keeping business logic and presentation code free of SQL and JDBC details. For example, a UserDAO class contains methods like findById(), save(), and delete() that execute SQL queries. The controller or service only calls DAO methods and works with plain Java objects, never with ResultSets or PreparedStatements directly. This makes it easy to swap the database, mock the DAO in tests, and keep each class focused on a single responsibility.

6

Describe the main components of the MVC (Model-View-Controller) pattern as applied in Java Servlets and JSP.

Identify which part handles user input, business logic, and output.

MODEL ANSWER

The Model holds application data and business rules — typically a JavaBean or entity class. The View is the JSP page that renders the HTML response the user sees. The Controller is the Servlet that receives the HTTP request, calls the right service or DAO to process data, sets attributes on the request/session, and forwards to the appropriate JSP view. This separation means UI designers can work on JSP without touching Java logic, and developers can update business rules without touching the view.

7

In a simple user management app, how would the UserDAO class interact with the controller servlet?

Think about method calls for user data retrieval or update.

MODEL ANSWER

The controller servlet receives an HTTP request — for example, to fetch user details by ID. It reads the 'id' parameter from the request, creates or injects a UserDAO instance, and calls 'UserDAO.getUserById(id)'. The DAO executes the SQL query and returns a User object (or null if not found). The controller stores the User as a request attribute ('request.setAttribute("user", user)') and forwards to the user-detail JSP for rendering. The controller orchestrates; the DAO handles data access; the JSP handles display.

8

Walk through the flow of a login operation using the MVC + DAO pattern in a Java web app.*Start from form submission and end with the user being logged in or shown an error.***MODEL ANSWER**

The user submits a login form via POST to LoginServlet (Controller). The servlet reads the username and password parameters, calls 'UserDAO.findByUsernameAndPassword(username, password)' (DAO). The DAO runs a PreparedStatement query against the database and returns a User object or null (Model). If a User is returned, the servlet stores it in HttpSession ('session.setAttribute("loggedUser", user)') and redirects to DashboardServlet. If null, it sets an error message attribute and forwards back to the login JSP (View).

9

What are the main advantages of using abstraction and reusability in the DAO layer?*Consider maintainability and flexibility for future changes.***MODEL ANSWER**

Defining a DAO interface (e.g., IUserDAO) and an implementation (UserDAOImpl) means the rest of the application depends on the interface. If you switch from MySQL to PostgreSQL, only the DAO implementation changes. For unit testing, you inject a mock DAO without needing a real database. Reusability means the same DAO methods (getUserById, getAllUsers) are called from multiple controllers without duplicating SQL. This keeps the codebase DRY and makes future changes safer and more localised.

10

How does the MVC + DAO pattern provide a foundation for frameworks like Spring MVC?*Think about how responsibilities are separated and how this maps to Spring.***MODEL ANSWER**

Spring MVC formalises and automates the same MVC + DAO separation you implement manually. @Controller replaces the Servlet, @Service holds business logic, @Repository replaces the DAO class, and Spring's dependency injection wires them together automatically. Spring also provides JdbcTemplate (replacing manual JDBC) and Spring Data JPA (replacing custom DAOs). Understanding MVC + DAO in raw Servlets makes it clear why Spring annotations exist and what problem each one solves.

11

Why is session tracking needed in Java web applications?*Focus on the stateless nature of HTTP and user-specific data.***MODEL ANSWER**

HTTP is a stateless protocol — each request is independent with no memory of previous requests. Without session tracking, the server cannot tell that two requests came from the same user. Session tracking lets you maintain state across multiple requests, such as keeping a user logged in, storing a shopping cart, or preserving multi-step form data. Without it, every page load would require the user to re-authenticate, making complex workflows impossible.

12

Explain how the HttpSession API works for session management in servlets.*Mention session creation, attribute storage, and session invalidation.***MODEL ANSWER**

When a client makes a request, you call `'request.getSession()'` to get or create an `HttpSession` object, which the server identifies with a unique session ID sent to the browser as a cookie (JSESSIONID). You store user-specific data with `'session.setAttribute("key", value)'` and retrieve it with `'session.getAttribute("key")'`. On logout, call `'session.invalidate()'` to destroy the session and remove all stored data. Sessions also expire after a configurable timeout (default 30 minutes in most servers).

13

Compare session tracking using cookies and hidden form fields.*Think about where data is stored and visibility to the client.***MODEL ANSWER**

Cookies store a session identifier in the browser and are sent automatically with every request to the same domain — they are invisible to the user but accessible via browser developer tools. Hidden form fields embed data in the HTML of every page and pass it back with each form submission — visible in page source and only work for form submissions, not for all requests. Cookies are the standard approach; hidden fields are a fallback when cookies are disabled, but they are cumbersome and expose data in page source.

14

What are some best practices for session security in Java web applications?*Consider session hijacking, timeouts, and secure cookie flags.***MODEL ANSWER**

Regenerate the session ID after login to prevent session fixation attacks (`'request.changeSessionId()'`). Set the `HttpOnly` flag on the session cookie to prevent JavaScript from reading it, and `Secure` flag to send it only over HTTPS. Configure a short session timeout for inactive sessions. Never store sensitive data like passwords in the session. Validate that the session belongs to the requesting user on every protected page. Use CSRF tokens to prevent cross-site request forgery on state-changing operations.

15

How does `session.invalidate()` differ from JWT-based token expiry?*Compare server-side and stateless client-side session management.***MODEL ANSWER**

`session.invalidate()` destroys the session on the server — immediately and completely. Even if the client still holds the JSESSIONID cookie, the server no longer recognises it. JWT (JSON Web Token) is stateless — the token lives on the client, and the server trusts it if the signature is valid and the expiry claim (`exp`) has not passed. To 'revoke' a JWT before expiry, you need extra infrastructure like a token blacklist. `HttpSession` is simpler to revoke; JWT scales better because the server stores no session state.

16

Describe how you would integrate a Bootstrap HTML template with a Java Servlet-based web application.

Focus on folder structure and linking static resources.

MODEL ANSWER

Place the Bootstrap CSS, JS, and image files inside the project's web root — typically in a 'static' or 'resources' folder alongside your WEB-INF directory. In your JSP files, link to them using relative paths: `<link rel="stylesheet" href="${pageContext.request.contextPath}/css/bootstrap.min.css">`. The context path prefix ensures links work regardless of the deployment context. Configure Tomcat or web.xml to serve static files directly. Dynamic content comes from JSP/Servlets; Bootstrap just controls the visual styling.

17

How do you separate static resources like CSS, JS, and images from dynamic content in a Java web app?

Think about the recommended folder structure.

MODEL ANSWER

Keep static files (CSS, JS, images, fonts) in dedicated folders at the web root level: `/webapp/css/`, `/webapp/js/`, `/webapp/images/`. Place JSP files either at the web root or in `WEB-INF/views/` (to prevent direct URL access). Dynamic Servlet endpoints handle business logic. Tomcat's `DefaultServlet` serves static files directly without invoking any Servlet code. This separation keeps responsibilities clear: static files are CDN-friendly and can be cached aggressively, while dynamic endpoints handle user-specific data.

18

What is the role of JSP, JSTL, and EL in rendering dynamic content in Java web applications?

Consider how data is passed from the backend to the view.

MODEL ANSWER

JSP (JavaServer Pages) lets you embed Java logic inside HTML to produce dynamic output. EL (Expression Language) provides a cleaner syntax to access data set as request or session attributes — for example, `${user.name}` instead of Java scriptlet code. JSTL (JSP Standard Tag Library) provides tags for common operations like loops (`<c:forEach>`), conditionals (`<c:if>`), and URL building (`<c:url>`), replacing messy Java code inside JSP. Together they keep the view layer mostly HTML with minimal server-side logic.

19

If you are building a professional-grade web app, what folder structure would you follow for your templates and static files?

Think about organizing files for maintainability and scalability.

MODEL ANSWER

A typical structure: `src/main/webapp/` as the root, with `WEB-INF/views/` for JSP templates (preventing direct browser access), `WEB-INF/web.xml` for configuration, `static/css/`, `static/js/`, `static/images/` for assets, and `WEB-INF/lib/` for JAR files. Java source in `src/main/java/` split by package: controller, service, dao, model. This follows Maven conventions and maps directly to Spring Boot's directory structure, making the transition to Spring seamless.

20

How would you handle file uploads in a Java Servlet using MultipartConfig?

Mention annotation/configuration and processing uploaded files.

MODEL ANSWER

Annotate the Servlet class with '@MultipartConfig(maxFileSize = 1048576)' to tell the container to parse multipart form data. The upload form must use 'enctype="multipart/form-data"'. In the servlet, call 'Part filePart = request.getPart("file")' to get the uploaded file. Retrieve the filename with 'filePart.getSubmittedFileName()', then save it with 'filePart.write(uploadDir + filename)'. Always validate file type and size before saving, and generate a unique filename (e.g., UUID) to prevent overwriting existing files.

21

What are the pros and cons of saving uploaded files as blobs in the database versus saving file paths?

Consider storage efficiency, backup, and retrieval performance.

MODEL ANSWER

Storing as blobs keeps files and their metadata together, simplifying backup and preventing broken links. However, large blobs bloat the database, slow down queries on unrelated tables, and bypass the web server's efficient static-file serving. Storing file paths in the database and files on disk (or cloud storage like S3) keeps the database lean, serves files efficiently via a CDN, and scales cheaply. The tradeoff is that you must ensure the file system stays in sync with the database. For most web apps, the file-path approach is preferred.

22

Write a code snippet to handle image upload and save the file to a specific folder on the server using Java Servlets.

Use MultipartConfig and getPart().

MODEL ANSWER

```
'Part filePart = request.getPart("profileImage"); String fileName = UUID.randomUUID() + "_" +
filePart.getSubmittedFileName(); String uploadPath = getServletContext().getRealPath("/uploads/");
filePart.write(uploadPath + fileName); response.getWriter().println("Uploaded: " + fileName);'. getRealPath
maps the '/uploads/' virtual path to the physical directory. Generating a UUID prefix prevents filename
collisions. Always check MIME type ('filePart.getContentType()') to reject non-image uploads before saving.
```

23

How would you implement a download servlet to allow users to download files they previously uploaded?

Focus on setting response headers and streaming file content.

MODEL ANSWER

Set 'response.setContentType("application/octet-stream")' to trigger a file download dialog, and 'response.setHeader("Content-Disposition", "attachment; filename=\"" + fileName + "\"') to suggest the filename. Then read the file from disk with a FileInputStream and write it to 'response.getOutputStream()' using a byte buffer. Always validate that the requested filename belongs to the logged-in user before serving it, to prevent unauthorised file access (path traversal attack).

24

Explain the basic concept of SMTP and how it is used with the JavaMail API.*Think about sending emails from a web application.***MODEL ANSWER**

SMTP (Simple Mail Transfer Protocol) is the standard protocol for sending emails between mail servers. JavaMail API provides classes to compose and send email messages over SMTP from a Java application. You create a Session with SMTP server properties (host, port, auth=true, starttls), create a MimeMessage with recipients, subject, and body, and call Transport.send(message) to dispatch it. Common SMTP providers for apps include Gmail (port 587), SendGrid, or Amazon SES.

25

Describe how you would send an HTML email with an attachment using JavaMail.*Mention message composition, multipart content, and transport.***MODEL ANSWER**

Create a MimeMessage and set To, From, and Subject. Build a MimeMultipart object. Add a MimeBodyPart with 'setContent(htmlString, "text/html")' for the HTML body. Add a second MimeBodyPart and call 'attachmentPart.attachFile(new File(filePath))' for the attachment. Set the multipart as the message content with 'message.setContent(multipart)'. Finally call 'Transport.send(message)'. Wrap in try-catch for MessagingException and use app-specific passwords or OAuth2 when authenticating with Gmail.

26

What is the difference between transactional and marketing emails?*Consider the purpose and content of each type.***MODEL ANSWER**

Transactional emails are triggered by a specific user action and contain information relevant to that individual — registration confirmations, OTPs, password resets, or payment receipts. They are expected and have high open rates. Marketing emails are sent in bulk to a list of subscribers to promote products or services — newsletters, discount announcements, or campaigns. Transactional emails should be sent through a reliable API (SES, SendGrid) with low latency; marketing emails require unsubscribe links and CAN-SPAM/GDPR compliance.

27

How would you generate a random OTP in Java for an email verification system?*Think about using the Random class or SecureRandom.***MODEL ANSWER**

'int otp = 100000 + new SecureRandom().nextInt(900000);'. SecureRandom is preferred over Random for security-sensitive values because it uses a cryptographically strong algorithm. This generates a 6-digit OTP between 100000 and 999999. Convert to String for storage and comparison. For a 4-digit OTP, use nextInt(9000) + 1000. Always use SecureRandom (not Math.random()) for any value that controls access, since predictable random values can be exploited.

28

Where would you store an OTP temporarily for verification — session or database? Why?*Consider data lifetime and security.***MODEL ANSWER**

For a single-server web app, storing the OTP in the HttpSession is simple and sufficient — it is already tied to the user's session and disappears on timeout or invalidation. For a multi-server deployment behind a load balancer, sessions may not be shared across servers, so storing the OTP in the database (with the user ID, hashed OTP value, and expiry timestamp) is more reliable. Always store a hash of the OTP rather than the plain value, and delete the record immediately after successful verification.

29

Describe how you would implement OTP expiry and what best practices you would follow.*Focus on time limits and handling expired OTPs.***MODEL ANSWER**

When generating the OTP, record the current time. On verification, check that the current time minus the generation time is within the allowed window (e.g., 5 minutes). If the window has passed, reject the OTP and optionally prompt the user to request a new one. Best practices: limit to one active OTP per user at a time (invalidate old ones when a new one is requested), limit verification attempts (lock after 3 wrong attempts), and never log plain OTP values. A database column 'otp_expires_at TIMESTAMP' makes expiry checks straightforward.

30

How can you use ChatGPT or LangChain4J to auto-generate an OTP logic function, and what would you check before using the code in production?*Think about code review, testing, and security.***MODEL ANSWER**

You can prompt ChatGPT to generate a Java method that creates an OTP, stores it with an expiry, and validates it. LangChain4J can be used to build a chain that generates the OTP and calls a JavaMail method to send it. Before using in production, verify that SecureRandom is used (not Math.random()), that the OTP is hashed before storage, that expiry logic handles time zones correctly, and that rate-limiting is in place. Write unit tests covering the happy path, expired OTP, and wrong OTP cases before deploying.

31

What is a payment gateway, and why do we need to integrate one (like Razorpay or Stripe) in a Java web application?*Focus on handling online transactions securely.***MODEL ANSWER**

A payment gateway is a third-party service that securely processes online payments — capturing card/UPI details, communicating with banks, and returning success or failure status. Integrating one is necessary because handling raw card data yourself requires PCI-DSS certification, which is complex and costly. Razorpay and Stripe handle all sensitive payment data on their servers; your application only receives a secure token or order ID. This shifts the security burden to the gateway and reduces your compliance overhead.

32

Describe the flow of integrating a payment button (e.g., Razorpay) with a Spring Boot backend.

Mention front-end, API calls, and backend verification.

MODEL ANSWER

The frontend calls your Spring Boot backend to create a Razorpay order ('POST /api/create-order') passing amount and currency. Your backend calls the Razorpay API using your secret key and returns the order_id to the frontend. The Razorpay JavaScript SDK opens the payment modal; on successful payment it returns payment_id, order_id, and a signature. Your frontend sends these to 'POST /api/verify-payment'. Your backend verifies the signature using HMAC-SHA256 with your secret key, confirms payment status, and updates the order in your database.

33

How do you securely manage API keys when integrating payment gateways?

Consider environment variables, config files, and not exposing keys in code.

MODEL ANSWER

Never hardcode API keys in source code — they would be exposed in version control. Instead, read them from environment variables ('System.getenv("RAZORPAY_KEY_SECRET")') or from application.properties using '@Value("\${razorpay.key}")'. In production on cloud platforms, inject secrets via environment configuration or a secrets manager (AWS Secrets Manager, Vault). Add the .env or properties file to .gitignore. Rotate keys immediately if you suspect a leak. Use separate keys for development and production environments.

34

What is webhook verification, and why is it important in payment gateway integrations?

Focus on confirming payment status and preventing fraud.

MODEL ANSWER

A webhook is an HTTP POST from the payment gateway to your server notifying you of a payment event (success, failure, refund). Webhook verification is the process of confirming that the payload genuinely came from the gateway and was not tampered with. Razorpay sends a signature header computed using HMAC-SHA256 over the payload with your webhook secret. Your server recomputes the signature and compares — if they match, you trust the event. Without this, an attacker could send a fake 'payment successful' payload to your server and unlock goods without paying.

35

How could you use ChatGPT to help you test payment API response payloads, and what should you verify before trusting the AI's output?

Think about data accuracy and matching actual gateway documentation.

MODEL ANSWER

ChatGPT can generate sample JSON payloads for Razorpay success, failure, and webhook events, which you can use to write unit tests for your payment verification logic without hitting real APIs. It can also draft test cases covering edge cases like payment captured vs. authorised-only. Before using the payloads, cross-check every field name and value against the official Razorpay API documentation, since AI may hallucinate field names or signature formats. Always run your tests against the actual sandbox environment to confirm real-world behaviour.

36

What is AJAX, and how does it improve the user experience in web applications?*Consider page reloads and asynchronous data updates.***MODEL ANSWER**

AJAX (Asynchronous JavaScript and XML) allows the browser to send HTTP requests and receive responses in the background without reloading the whole page. This makes web applications feel faster and more interactive — for example, submitting a form, checking if a username is available, or loading more records all happen without the jarring full-page reload. The browser updates only the relevant part of the DOM with the new data. Modern AJAX uses JSON (not XML) and the Fetch API or Axios instead of the original XMLHttpRequest.

37

How would you use AJAX to check if an email already exists before allowing user registration in a Java web app?*Mention client-side request and server-side validation.***MODEL ANSWER**

On the registration form, attach an 'onblur' event to the email field. The handler calls `fetch("/check-email?email=" + email).then(res => res.json()).then(data => { if (data.exists) showError("Email taken"); })`. The server-side `CheckEmailServlet` queries the database and returns `{'exists': true/false}` as JSON. This gives the user instant feedback without a page reload. Always repeat the validation server-side during the actual registration POST, since client-side checks can be bypassed.

38

Write a simple JavaScript `fetch()` call that submits a form asynchronously to a servlet endpoint.*Use `fetch` with method `POST` and handle the response.***MODEL ANSWER**

`fetch("/register", { method: "POST", headers: { "Content-Type": "application/json" }, body: JSON.stringify({ name: nameInput.value, email: emailInput.value }) }).then(res => res.json()).then(data => { if (data.success) window.location = "/dashboard"; else showError(data.message); }).catch(err => console.error("Error:", err));`. The catch block handles network errors. The servlet reads the JSON body using `request.getReader()` and responds with JSON, keeping the interaction clean without form submission and page reload.

39

How does the concept of AJAX relate to using Axios in a React-based project?*Think about asynchronous HTTP requests in both cases.***MODEL ANSWER**

Both AJAX (via Fetch) and Axios perform asynchronous HTTP requests from the browser to a server. Axios is a library that wraps the browser's Fetch/XMLHttpRequest with a cleaner API, automatic JSON parsing, request/response interceptors, and better error handling. In React, you use Axios (or Fetch) in a `useEffect` hook to call your Spring Boot REST API and update component state with the response. The underlying idea is identical to AJAX in a Servlet app — send a request, get data, update the UI without a full reload.

40

What is CI/CD, and why is it important for modern Java application development?*Focus on automation, faster releases, and reliability.***MODEL ANSWER**

CI (Continuous Integration) is the practice of automatically building and testing code every time a developer pushes changes, catching integration issues early. CD (Continuous Delivery/Deployment) extends this by automatically deploying the tested build to staging or production. For Java applications, CI/CD eliminates manual build steps, ensures tests always run before deployment, reduces the risk of human error, and enables teams to release multiple times a day confidently. Without CI/CD, deployments are slow, error-prone, and stressful events.

41

Describe the typical steps in a CI/CD pipeline for a Java web application.*Mention build, test, and deploy stages.***MODEL ANSWER**

A typical pipeline includes: (1) Source — code pushed to Git triggers the pipeline. (2) Build — Maven or Gradle compiles the code and packages it as a JAR/WAR. (3) Test — unit and integration tests run; the pipeline fails if any test fails. (4) Static Analysis — tools like SonarQube check code quality. (5) Containerise — a Docker image is built and tagged. (6) Deploy to Staging — the image is deployed to a staging server for smoke testing. (7) Deploy to Production — after approval (or automatically for CD), the image is deployed to production.

42

How would you set up a simple GitHub Actions workflow to build and deploy a Spring Boot app?*Think about YAML configuration and deployment steps.***MODEL ANSWER**

Create `.github/workflows/deploy.yml`. Define `'on: push: branches: [main]'`. Under jobs, add a 'build' job running on 'ubuntu-latest'. Steps: checkout code ('actions/checkout@v3'), set up JDK ('actions/setup-java@v3' with version 17), run 'mvn clean package -DskipTests', build Docker image ('docker build -t myapp:\${{ github.sha }} .'), push to Docker Hub ('docker push'), and SSH into the deployment server to pull and restart the container. Store credentials as GitHub Secrets to avoid exposing them in the workflow file.

43

What is the role of automated testing (e.g., JUnit) in a CI/CD pipeline?*Consider catching bugs before deployment.***MODEL ANSWER**

Automated tests act as a quality gate in the pipeline — if any test fails, the pipeline stops and the broken code is never deployed. JUnit unit tests verify individual methods; integration tests (e.g., with Spring's MockMvc) verify controller and service interactions; and end-to-end tests verify full user flows. Running these on every push means bugs are caught within minutes of introduction, at the lowest cost to fix. Without automated tests in CI, you are essentially deploying untested code and relying on production to find bugs.

44

Integrative: How would you combine the DAO pattern, session management, and AJAX to create a secure and responsive login system in a Java web application?

Think about backend separation, user authentication, and real-time feedback.

MODEL ANSWER

The login form submits credentials via an AJAX POST to LoginServlet. The servlet calls `UserDAO.findByCredentials()` — DAO handles the SQL with a `PreparedStatement` (preventing SQL injection). If a `User` is returned, the servlet creates an `HttpSession`, stores the user, regenerates the session ID (preventing session fixation), and returns JSON `'{"success": true}'`. If not, it returns `'{"success": false, "message": "Invalid credentials"}'`. The JavaScript updates the UI immediately without a reload. This combines DAO (data access), session management (state), and AJAX (UX) cleanly.

45

Integrative: If you are asked to build a registration system that sends an OTP email and supports file upload for profile pictures, which patterns and technologies from this module would you use, and how would you organize the flow?

Consider DAO, JavaMail, session, and file upload handling.

MODEL ANSWER

`Controller` (`RegistrationServlet`) handles the multipart POST. It extracts form fields and delegates to `UserDAO.createUser()` for database persistence (DAO pattern). The `MultipartConfig` annotation lets it call `request.getPart('profilePic')` and save the file to the uploads folder with a UUID name. After saving the user, it generates an OTP via `SecureRandom`, stores it in the session (or database with expiry), and calls `EmailService.sendOTP(user.getEmail(), otp)` using JavaMail over SMTP. The user is then forwarded to an OTP verification JSP. Each concern sits in its own class.

46

Integrative: How can you ensure that integrating external APIs (like payment gateways) does not break your deployment pipeline, and what tools from this module would help you automate testing and deployment?

Think about CI/CD, automated tests, and API mocking.

MODEL ANSWER

Write JUnit tests that mock the Razorpay API responses using a mocking framework (Mockito), so tests do not make real network calls and always pass in CI. Add a contract test that verifies your signature-verification logic against a known payload and key. Include these in the Maven test phase, ensuring the GitHub Actions pipeline blocks deployment if any payment logic test fails. Use environment-variable secrets for API keys in the pipeline. Deploy to a staging environment first, run a smoke test against the Razorpay sandbox, and only promote to production after the staging stage passes.